

פתרון -שאלה 2 (תקינות נתונים), מועד ב' 2026

a.

הסכמה הנתונה מאפשרת לשייך עובד למחלקה (dept_emp) ולציין מי מנהל מחלקה (dept_manager). בכל סעיף אני כותב תחילה את הפתרון בתוך הסכמה הקיימת, ואז את יתרונותיו וחסרונותיו; הרחבה מוצעת רק כשהסכמה הקיימת אינה מספיקה.

i.

הפתרון: המנכ"ל משויך למחלקת הנהלה (מחלקה רגילה המייצגת את ההנהלה הראשית) כחבר ב-dept_emp וכמנהל ב-dept_manager.
יתרון: הכול בתוך הסכמה הקיימת, בלי שינוי מבנה; המנכ"ל הוא עובד רגיל במחלקה רגילה.
חסרון: מחלקת ההנהלה אינה נבדלת מבנית ממחלקה תפעולית - ההבחנה נשענת על שמה בלבד.

ii.

הפתרון: סמנכ"ל משויך למחלקה שהוא מנהל — סמנכ"ל הכספים הוא חבר (dept_emp) ומנהל (dept_manager) של מחלקת הכספים.
יתרון: dept_emp נשארת מצבת כוח אדם מלאה, כי כל עובד — כולל מנהל — יושב ביחידתו.
חסרון: אין שכבת "הנהלה" מפורשת; מיהו מנהל נגזר מ-dept_manager ואינו מוחזק כישות נפרדת.

פתרון אלטרנטיבי — שיוך כפול: המפתח של dept_emp הוא (emp_no, dept_no), ולכן אותו עובד יכול להשתייך לשתי מחלקות. משייכים את סמנכ"ל הכספים גם למחלקת הכספים וגם למחלקת ההנהלה, וכך נשמר במפורש שהוא חבר בשתי השכבות. יתרון: השתייכותו להנהלה גלויה בנתונים. חיסרון: ספירת עובדים תמימה סופרת אותו פעמיים, ויש להחליט מהו השיוך הראשי.

iii.

בתוך הסכמה הקיימת: אפשר לקודד את ההיררכיה בשם המחלקה - 'Sales', 'Sales/Europe', 'Sales/Europe/France' - בלי שינוי מבנה. יתרון: מייד. חיסרון: השם אינו קשר אמיתי, אי אפשר לאכוף אותו או לשלוף באמינות את כל תת-היחידות של מכירות.

הרחבה עדיפה: עמודת אב parent_dept_no (מפתח זר לאותה טבלה) ההופכת את department לעץ:

```
ALTER TABLE departments
```

```
ADD COLUMN parent_dept_no CHAR(4) NULL,
```

```
ADD FOREIGN KEY (parent_dept_no) REFERENCES departments (dept_no);
```

כל מחלקה מצביעה על אביה, וקריאת רמה היא self-JOIN
(מכירות → אירופה → צרפת = שני self-JOIN):

```
SELECT s.dept_name AS sales, e.dept_name AS europe, f.dept_name AS france
```

```
FROM departments AS s
```

```
JOIN departments AS e ON e.parent_dept_no = s.dept_no
```

JOIN departments **AS** f **ON** f.parent_dept_no = e.dept_no
WHERE s.dept_name = 'Sales';

יתרון: קשר אמיתי הניתן לאכיפה, בעומק כרצוננו.
חסרון: שאילתה מציינת רמה אחר רמה; רק עומק בלתי-מוגבל היה מצריך WITH RECURSIVE (מעבר לחומר הקורס).

b.

בתוך הסכמה הקיימת: התפקיד נשמר כמחרוזת חופשית ב-titles.title.
יתרון: פשוט, בלי JOIN, ותפקיד חדש נכנס מיד.
חסרון, לפי שלושת המקרים בשאלה:
 (i) אותו תפקיד נכתב בכמה נוסחים ובשגיאות כתיב, ולכן ספירה או השוואה לפי תפקיד אינה אמינה;
 (ii) אין דרך למיין לפי בכירות, כי סדר מחרוזות אינו סדר בכירות;
 (iii) אין אימות שהתפקיד מתאים למחלקה.

הרחבה מוצעת - קטלוג תפקידים: מחליפים את המחרוזת ב-title_id title המצביע על טבלת ייחוס:

job_titles (title_id PK, title_name **UNIQUE**, family, seniority_level)
 title_aliases (alias PK, title_id FK)
 dept_allowed_titles (dept_no FK, title_id FK, **PRIMARY KEY** (dept_no, title_id))

i.

title_aliases ממפה כל נוסח או כינוי אל ה-title_id הקנוני; הנשמר תמיד הוא ה-title_id, וכל הנוסחים מתלכדים לתפקיד אחד.

ii.

seniority_level מספרי בתוך family נותן סדר בכירות אמיתי (זוטר > בכיר > אלוף), במקום סדר מחרוזות שבו 'Senior X' מקדים את 'X'.

iii.

dept_allowed_titles מתירה לכל מחלקה קבוצת תפקידים. CHECK אינו יכול לאכוף זאת - האילוץ מערב שלוש טבלאות והוא תלוי-זמן - ולכן אכיפה בטריגר או באפליקציה.

יתרון ההרחבה: אוצר מילים מבוקר, מיון בכירות, ואימות מול המחלקה.
חסרון: מורכבות גבוהה יותר ו-JOIN בכל שליפה.

c.

יחס ראשון-hire_date < birth_date

1. עובד נולד לפני שהתחיל לעבוד; מחמיר יותר, $16 + \text{hire_date} \geq \text{birth_date}$ שנים.

2. כל חישוב ותק וגיל נשבר, ועובד מתחת לגיל העסקה חוקי הוא חשיפה רגולטורית.

3. טבלה אחת, שתי עמודות באותה שורה: (CHECK (birth_date < hire_date)).

on שני - from_date < to_date

1. תקופה באורך חיובי, ב-dept_emp, salaries, titles, dept_manager ו-dept_manager. > ולא => , כי הייצוג הוא קטע חצי-פתוח.
2. שורה באורך אפס בלתי נראית לשאילתה נקודתית אך נראית ל-COUNT, ולכן "כמה עובדים" ו"מי העובדים" יחזירו תשובות סותרות.
3. טבלה אחת: CHECK בכל אחת מארבע הטבלאות. תחילה צריך לנקות את השורות המפרות, כי ALTER נדחה כל עוד קיימת אחת.

on שלישי - salaries.from_date >= employees.hire_date

1. תקופות השכר, התפקיד והמחלקה נמצאות בתוך תקופת ההעסקה.
2. עלות שכר מחושבת על שנים שבהן העובד לא הועסק, ואין מקור אמת יחיד למועד תחילת ההעסקה.
3. שתי טבלאות, ולכן CHECK אינו אפשרי - הוא אינו יכול להכיל תת-שאילתה. הפתרון הוא טריגר BEFORE INSERT ו-BEFORE UPDATE שקורא את hire_date ומרים SIGNAL, בליווי שאילתת ביקורת, כי טריגר אינו מגן על שורות קיימות.

d. (בונוס)

1.

כן. אין זה ערך שגוי בשדה אלא הפרה של זהות הישות - אותו אובייקט קיבל כמה מפתחות ראשיים. אי אפשר לתקן בעדכון תא בודד, אך כן ניתן לתקן על ידי מחיקת המופעים העודפים (סעיף 3). השלכות: סרט שמופיע שלוש פעמים מקבל משקל פי שלושה מהראוי לו בכל צבירה; מודל ההמלצות רואה כמה פריטים במקום אחד, וסביר שכל עותק ימליץ על האחרים (אותו במאי, אותם שחקנים); וחלוקה אקראית ל-train ו-test תשים מופע בכל צד - דליפת נתונים (data leakage), שכן המודל נבחן על סרט שכבר ראה באימון.

2.

כן, זו בעיית נתונים. אמנם כל דירוג בנפרד הוא תצפית תקפה - הסטודנט דירג בכנות שני פריטים שנראו לו שונים - אך משום שאיננו יודעים שאלה דירוגים של אותו אובייקט, איננו יכולים לנצל את הפער ביניהם (סעיף 3), והצבירה מעוותת.

מניעה בלי מודעות לכפילות: UNIQUE (student_id, movie_id) לא היה מונע, כי המזהים שונים והאילוץ מרובה. אילוץ ייחודיות על המפתח הטבעי של הסרט, UNIQUE (name, year), כן היה מונע, והוא נכתב משיקולי מידול רגילים בלי מודעות לכפילות. הסתייגות: יש רימייקים וסרטים שונים באותו שם ובאותה שנה, ולכן בייצור מרחיבים את המפתח.

3.

בוחרים מזהה קנוני לכל סרט (המזהה הקטן ביותר לכל (name, year), ומכאן שני פתרונות:

עדיף - מחיקה והפניה למקור: מפנים את ratings.movie_id מהעותקים אל הקנוני ומוחקים את שורות העותק מ-movies; נשאר מזהה אחד לכל סרט. היכן שאותו מדרג דירג כמה עותקים, הפער בין דירוגיו הוא בדיוק אות היציבות - מודדים אותו לפני האיחוד ואז ממזגים (למשל בממוצע).

חלופה - טבלת מיפוי בלי מחיקה:

```
CREATE TABLE movie_canonical (  
  movie_id      INT PRIMARY KEY REFERENCES movies (id),  
  canonical_movie_id INT NOT NULL REFERENCES movies (id),  
  match_rule    VARCHAR(30) NOT NULL  
);
```

המפתחות הזרים הקיימים נשארים תקפים, הדירוגים ממשיכים להצביע על המזהה שאליו הוזנו, והפעולה הפיכה וניתנת לביקורת - יתרון כשהמיזוג אינו ודאי.

יציבות הדירוג: משאיחדנו את המופעים, דירוגים שונים של אותו אובייקט מאותו מדרג מאפשרים למדוד את יציבותו - פער גדול מעיד על אי-ודאות.

```
SELECT canonical_movie_id, AVG(spread) AS avg_disagreement  
FROM (SELECT r.student_id,  
  mc.canonical_movie_id,  
  MAX(r.rating) - MIN(r.rating) AS spread  
  FROM ratings AS r  
  JOIN movie_canonical AS mc ON mc.movie_id = r.movie_id  
  GROUP BY r.student_id, mc.canonical_movie_id  
  HAVING COUNT(*) > 1) AS per_rater  
GROUP BY canonical_movie_id  
ORDER BY avg_disagreement DESC;
```

ברמת הסרט, STDDEV של ממוצעי המופעים.

השאלתה עצמה מעבר למצופה מסעיף בונוס במבחן.
מחוץ לבחינה ניתן להעריך יציבות בשיטות שונות, כמו זו.